```
      functionToDebounce(...args)
    }, wait)
  }
}
```

Our actual submit logic is in the basicSubmit method. If the input field is blank, we reset; otherwise, we submit the form. The submit method from before, the one that is invoked by the form action, is just a function variable that wraps that basicSubmit inside a call to a new method called debounce.

The debounce function is a little confusing because it's treating a function as a variable. The basic idea here is that we take a function as an argument —functionToDebounce in the code—and put it on a delay so that it waits a certain amount of time before that function is actually called. However, we also set the timer up so that if debounce is invoked again during the delay time, the first delay is canceled and we start again with a new delay. The upshot is that the functionToDebounce is only invoked after debounce has not been triggered for whatever the time delay is, ideally long enough to prevent results from crossing each other.

The debounce function takes a function as an argument and returns a new function that wraps the argument in a debouncing timer. That returned function does two things when called: it clears the existing timeout and it sets up a new timeout. JavaScript's setTimeout function takes a callback argument, which is invoked after the delay time. Inside the callback, we clear the timeoutId and call the original argument function. However, if the returned function is called again, it calls clearTimeout, which cancels the existing timer and starts a new one. Again, the result is that the original argument function is only called after the event is not invoked for a set amount of time.

This is not the only way to do debouncing. In practice we'd probably want to make it a utility available to other controllers. Alternately, the Stimulus-Use package[65] provides debounce along with other useful utilities that are designed to be mixed in to Stimulus controllers.

That gives us some form behavior in Hotwire with Turbo and Stimulus. You can